MIRS Lab guides

Dr. Mauro Pazmino

2024-10-22

Table of contents

Preface **3**

1 How to use our scripts for mid infrared spectroscopy **4**
1.1 Opus dei 4
1.2 Bad blood 4
1.3 How to use the scripts 5
1.3.1 Using only jupyter notebook 5
1.3.2 Using a jupyter notebook and the .py files approach 5
1.4 Examples on how to use the opus_dei.py and bad_blood.py 6
1.5 Errors using bad blood 9

2 Visualization of spectroscopy data **10**
2.1 Basic plot with defaults 10
2.2 Improving the defaults 11
2.3 Plotting the means of different classes 13
2.4 Plotting all samples of different classes 14

3 Machine learning pipelines **16**
3.1 Basic exploratory machine learning analysis 16
3.1.1 Explore different models 16
3.1.2 Model optimization 18
3.1.3 Test the final model 19
3.2 Nested Cross validation 21
3.3 Nested cross validation plus validation on unseen data 30
3.3.1 Set plot aesthetics 31
3.3.2 Defining the fuctions 31
3.3.3 Split the data 36
3.3.4 Setting the parameters of nested cross validation 36
3.3.5 Setting the parameters of nested cross validation 37
3.3.6 Run the loop 37

4 A closer look into spectral quality **42**

References **43**

Preface

This document is a series of guides for students that are experimenting with with infrared spectrosocpy in the context of surveillance with vectors. Here, we will describe how to use scripts to extract data, clean it, assembling data sets for machine learning as well as some tutorials on how to plot this type of data.

This guide is based on the follwing published work:

- González Jiménez, Mario, et al. "Prediction of mosquito species and population age structure using mid-infrared spectroscopy and supervised machine learning." Wellcome open research 4 (2019).(González Jiménez et al. 2019)
- Siria, Doreen J., et al. "Rapid age-grading and species identification of natural mosquitoes for malaria surveillance." Nature Communications 13.1 (2022): 1501. (Siria et al. 2022).
- Pazmiño-Betancourth, Mauro, et al. "Evaluation of diffuse reflectance spectroscopy for predicting age, species, and cuticular resistance of Anopheles gambiae sl under laboratory conditions." Scientific Reports 13.1 (2023): 18499.(Pazmiño-Betancourth et al. 2023)
- Pazmiño-Betancourth, Mauro, et al. "Advancing age grading techniques for Glossina morsitans morsitans, vectors of African trypanosomiasis, through mid-infrared spectroscopy and machine learning." Biology Methods and Protocols 9.1 (2024): bpae058. (Pazmiño-Betancourth et al. 2024)

if you have any questions you can reach me or Ivan Casas

1 How to use our scripts for mid infrared spectroscopy

This is a quick tutorial on how to use the scripts for decoding files from the FTIR machine and to clean up and assembling all the files into a csv file.

> **! Important**
>
> These guides assume you have some knowledge about python such as installing/importing packages, building functions, working with jupyter notebooks, etc.

The scripts are:

- opus_dei
- bad_blood

1.1 Opus dei

This script takes the files from the FTIR machine that are proprietary (you can only read them with OPUS software) and transform them into .dpt (data point table) files and .mzz files.

Dpt files are basically a two column file with one column will have the wavenumbers value and the second column will have the absorbance. It can be opened in excel, notes, etc.

Mzz files are compressed files especially created for this project. They contain only the absorbance and can be opened by bad_blood script.

1.2 Bad blood

This script reads all your files either as .dpt or .mzz and assess the quality of each file (low intensity, atmospheric interference, and distortion by the anvil). Then, it will create a table with all the absorbance, and metadata that you can use for your machine learning analysis.

1.3 How to use the scripts

There are two approaches (maybe three) on how you can use them in your workflow. These are: 1. Using the jupyter notebook file only 2. Using a combination of jupyter notebook and the .py files

1.3.1 Using only jupyter notebook

This is self explanatory. You just need to copy the jupyter notebook into your working folder. Open it with jupyter or jupyterlab or vscode and follow the instructions.

1.3.2 Using a jupyter notebook and the .py files approach

This is the approach for intermediate/advance users. Combines the flexibility of jupyter notebooks plus the convenience of having all your functions in a separate file. This improves readability, maintainability of your code (and it looks more slick!) For this you will need to have a specific folder structure. Something like this:

In the folder called source code (src), you will put the bad_blood.py and opus_dei.py files. Then, you will create your jupyter notebook for your analyses. You need to include two important pieces of code.

First:

```
%load_ext autoreload
%autoreload 2
```

This code you will only need to run it once (at the beginning of your session). This allows you to change or improve your functions from the source code folder and they will be updated automatically.

The second code block you will need is this:

```
sys.path.append('/your/path/toyour/sourcecode/src')
```

You need to specify the path where bad blood and opus dei are located, so jupyter knows where to search when you import those functions. This can be added in the cell where you import all your packages.

> **Tip**
>
> Try to use a IDE such as Visual Studio Code from the beginning.

1.4 Examples on how to use the opus_dei.py and bad_blood.py

To run opus dei and bad blood you need the following packages:

You need to set up your jupyter notebook (this also work if you are only using jupyter)

```
# setting up the path to src folder
import sys
sys.path.append("/Users/mauropazmino/Documents/University/Workhops/How to use the scripts/

# the packages that you will use
import numpy
import pandas

# importing opus dei and bad blood functions
from bad_blood import bad_blood_v3_real
from opus_dei import opus_dei
```

Once you have imported your modules. You need to run opus_dei function to decode the files into readable format.

The opus dei function requires two parameters: the folder where the raw files are located and the folder where you want to save the new files.

```
raw_data_folder = "./data/raw_data"

processed_data_folder = "./data/processed_data"

# run the function
opus_dei(raw_data_folder, processed_data_folder)
```

```
Your files have been proceesed!!!
```

Once the files has been decoded, you will get a message.

After that, you can run the bad blood function to the processed files.

The bad blood function requires two parameters, one is the path where the processed files are located and two, the name of the assembled datafile. Moreover, this function requires all the files have the same file length.In our lab, we have the following filename convention:

- XX-XX-XX-XX-XX.dpt

where each part of the file that is separate by a hyphen represents some metadata.For example in the following file: UV-AG-F-NOU-R1.0.dpt

UV means UV experiment, AG means Anopheles gambiae, F meeans female, etc. It does not matter how many sections you have in your file name (2 from 100), as long as all your files have the same number of sections. If one file has less or more, the function will give you an error and it will indicate which files need to be fixed. You can do that on the .mzz files but it would be better if you do it in your raw files and then run opus dei again.

> ❗ File name length is important!
>
> Always check that your files have the same length.

Now you run the function bad_blood

```
processed_data_folder = "./data/processed_data"
name_data = 'lab_guide_sample'

bad_blood_v3_real(processed_data_folder,name_data)
```

```
  0%|          | 0/9 [00:00<?, ?it/s]

100%|      | 9/9 [00:00<00:00, 365.17it/s]



Assessing spectra with low quality..
    0 spectra have been discarded because their low intensity

Checking for abnormal background...
    0 spectra have been discarded because they were distorted by the anvil

Checking for spectra with atmospheric interferences..
    0 spectra have been discarded because have atmospheric interferences
```

```
Cat 1 - Options:
----------------
      UV
Cat 2 - Options:
----------------
      AG
Cat 3 - Options:
----------------
      F
Cat 4 - Options:
----------------
      NOU
Cat 5 - Options:
----------------
      R1
Cat 6 - Options:
----------------
      7
      4
      5
      3
      6
      0
      8
      2
      1

  0%|          | 0/9 [00:00<?, ?it/s]

100%|     | 9/9 [00:00<00:00, 490.97it/s]

This last process has lasted 0.02 s
The new matrix contains 9 spectra.
Your new matrix MIRS_lab_guide_sample_20241026 is in the folder data
```

As you can see, bad blood will read all your files and will do a quality control on each one of them. If a file does not pass the quality control, it will be discarded. After that, it will put all you files together with the metadata tied to them in a nice table format. The name of the final data set will include the date when it was created for future reference.

1.5 Errors using bad blood

Errors when using bad blood

Bad blood will give the following error:

```
UnboundLocalError: cannot access local variable 'tmp2'
where it is not associated with a value
```

if the folder where it reads the files does not exist or it does not have any mzz or dpt files on it.

If you want to know more what bad blood does, you can check Chapter 4 for more information

2 Visualization of spectroscopy data

This is a quick tutorial on how to plot and make publication-quality figures from infrared spectroscopy data using Python. We are using matplotlib, pandas, numpy and seaborn for this tutorial. Check the requierements file. The data used in this tutorial is from a small experiment where we collected spectra from mosquito thoraces using a Attenuated Total Reflection. There are two classes, mosquitoes exposed to UV and non exposed to UV.

2.1 Basic plot with defaults

First, we need to import the data that we are going to use. The data will be arranged with samples as rows and wavenumbers and categories as a columns.

```
# import packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sn
```

```
data = pd.read_csv("./data/UV_pilot.csv")
data.head()
```

	Specie	Sex	Exposed	Reeplicate	ID	4006	4004	4002	4000	3998	...	415
0	AG	F	YES	R1	2	0.00477	0.00462	0.00448	0.00434	0.00439	...	0.33421
1	AG	F	YES	R1	3	0.00596	0.00631	0.00629	0.00621	0.00623	...	0.32344
2	AG	F	YES	R1	8	0.00542	0.00513	0.00511	0.00517	0.00537	...	0.33420
3	AG	F	NOU	R1	1	0.00681	0.00690	0.00695	0.00683	0.00666	...	0.29993
4	AG	F	NOU	R1	3	0.00543	0.00552	0.00556	0.00555	0.00548	...	0.29425

We select the wavenumbers and their absorbance values and we call this "X". We also select the categorical values we want to plot, and call it "y_labels"

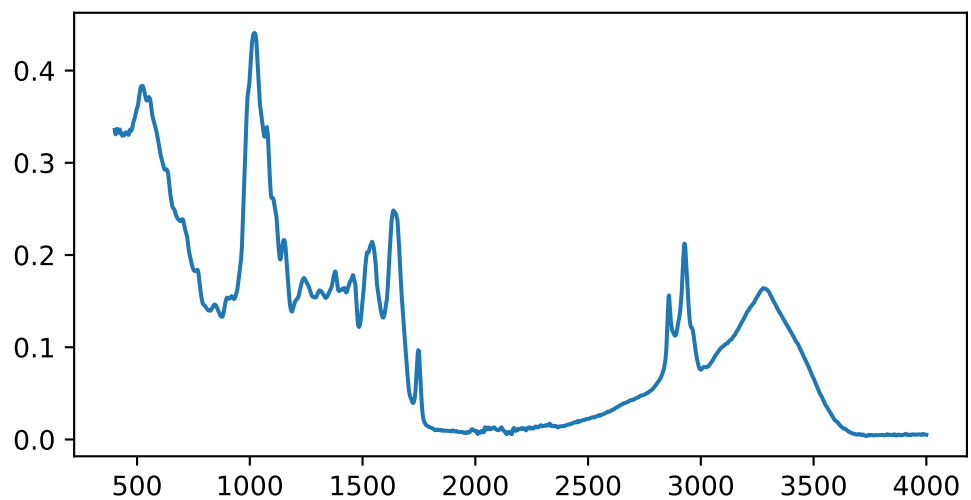
```
X = data.loc[:,"4000":'401']
y_labels = data.loc[:,"Exposed"]
```

To plot a line, we need x and y values. The values of x will be the specific wavenumbers (which are the names of the columns of X). We extract the names of the columns of X and put them on a list as a integers.

```
wnLabels = X.columns.values
waveNums = [int(x) for x in wnLabels]
```

Now that we have teh values of x, we need the absorbance of a sample. We can select any row from X.

```
fig, ax = plt.subplots(figsize=(6,3))
ax.plot(waveNums, X.iloc[2,:])
```

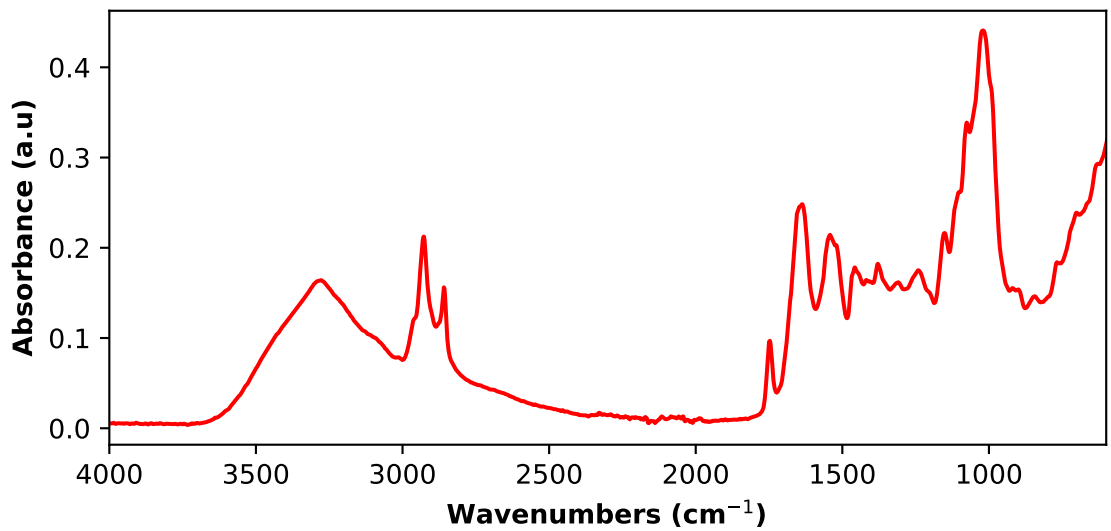


2.2 Improving the defaults

The defaults of matplotlib or seaborn are not that great. Increasing the linewidth and font size, choosing another color and add axis labels can help with the overall quality and readability of the plot. Moreover, infrared spectral data is plot from 4000 to 400 cm^{-1}. So, we need to reverse the x-axis.

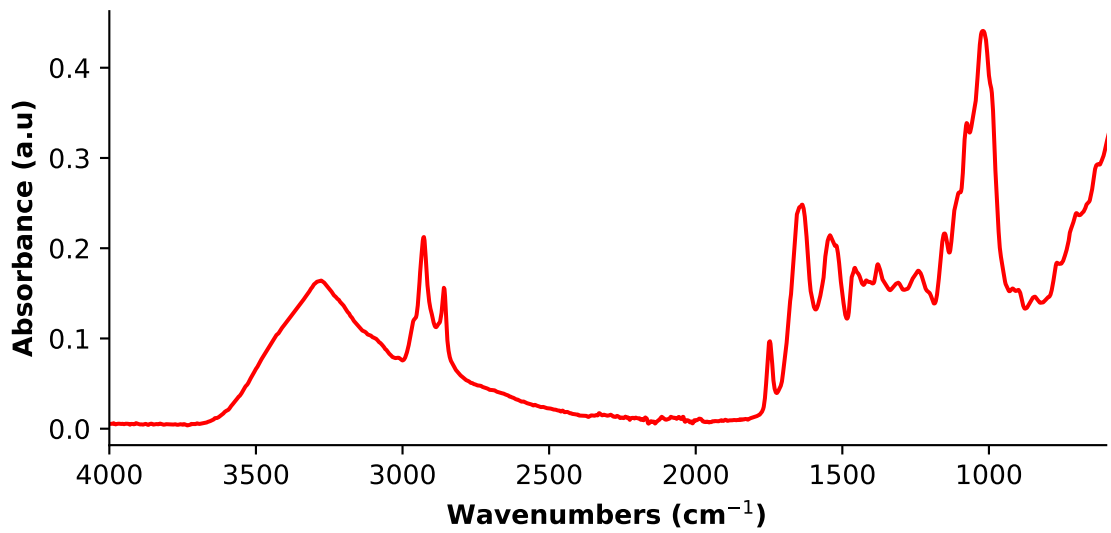
```
fig, ax = plt.subplots(figsize=(6,3), tight_layout=True)
ax.plot(waveNums,X.iloc[2,:],color='r',linewidth=1.5)

ax.set_xlim(4000, 600)
ax.set_ylabel("Absorbance (a.u)", fontweight='bold')
ax.set_xlabel('Wavenumbers (cm$^{-1}$)', fontweight='bold')
```

```
Text(0.5, 0, 'Wavenumbers (cm$^{-1}$)')
```



We can choose different aesthetics for the final plot like removing the spines.

```
fig, ax = plt.subplots(figsize=(6,3), tight_layout=True)
ax.plot(waveNums,X.iloc[2,:],color='r',linewidth=1.5)

ax.set_xlim(4000, 600)
ax.set_ylabel("Absorbance (a.u)", fontweight='bold')
ax.set_xlabel('Wavenumbers (cm$^{-1}$)', fontweight='bold')
ax.spines.right.set_visible(False)
ax.spines.top.set_visible(False)
```



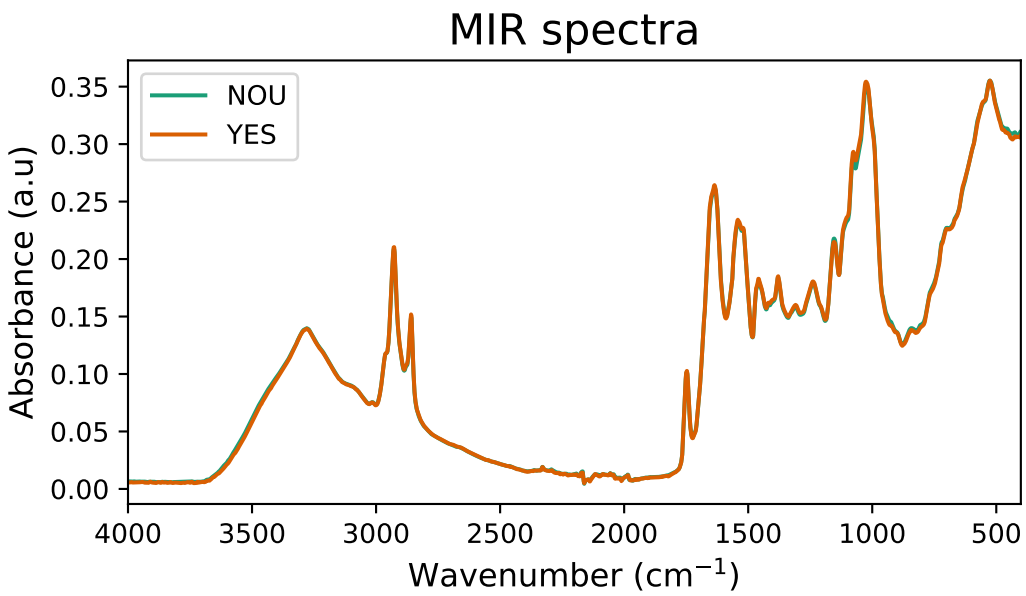
2.3 Plotting the means of different classes

To observe differences between classes, you might want to plot the mean spectra of each class you are assessing. The issue here is that the data is presented as rows, which complicates the plotting process since matplotlib like the samples in columns rather than rows.

The code is straightforward, and it works for any number of classes (as long as the number of colours is the same of greater than the number of classes)

```
# color hex codes can be used, as well as common color names. You can get more codes at ht

colors = ['#1b9e77','#d95f02','#7570b3','#e7298a','#66a61e']

# figure
fig, ax = plt.subplots(figsize=(6,3))


for i, c in zip(np.unique(y_labels), colors):
    sn.lineplot(x=waveNums, y=np.mean(X[y_labels == i], axis=0), label=i, color=c)
plt.legend()
ax.set_xlim(4000,401)
ax.set_title("MIR spectra",fontsize=16)
ax.set_xlabel("Wavenumber (cm$^{-1}$)",fontsize=12)
ax.set_ylabel("Absorbance (a.u)",fontsize=12)
```

```
Text(0, 0.5, 'Absorbance (a.u)')
```



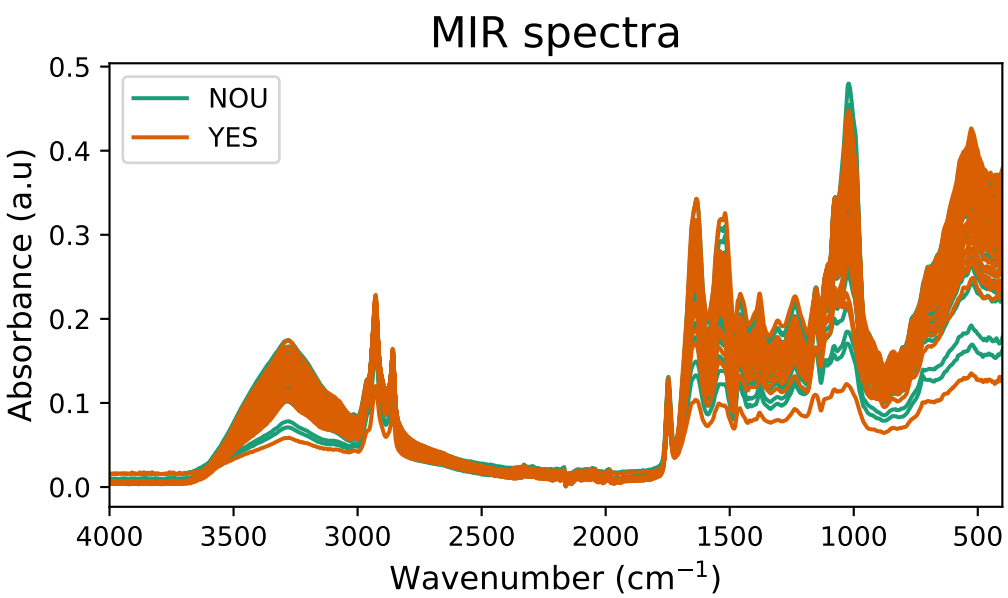
2.4 Plotting all samples of different classes

What if you want to plot not the means but all the samples? For this, we use the same approach as before, but instead of selecting all the samples and plot the mean, we select each sample and plot it. We use itertuples to go through them each row and plot it with the assigned colour. It might be a little bit slow if you have thousands of rows.

```
fig, ax = plt.subplots(figsize=(6,3))

# Iterates through each row and compare it to the label, and plot it.
for i, c in zip(np.unique(y_labels), colors):
    for row in X.loc[y_labels == i].itertuples(index=False):
        ax.plot(waveNums, row, color=c, label=i)

# Legend creation
handles, labels_2 = ax.get_legend_handles_labels()
newLabels, newHandles = [], []
for handle, label_1 in zip(handles, labels_2):
    if label_1 not in newLabels:
        newLabels.append(label_1)
```

```
        newHandles.append(handle)

ax.legend(newHandles, newLabels)
ax.set_xlim(4000,401)
ax.set_title("MIR spectra",fontsize=16)
ax.set_xlabel("Wavenumber (cm$^{-1}$)",fontsize=12)
ax.set_ylabel("Absorbance (a.u)",fontsize=12)
```

```
Text(0, 0.5, 'Absorbance (a.u)')
```



3 Machine learning pipelines

In this section, we are going to describe some of the pipelines that our group uses as a starting point in the analysis of infrared data. If you want to explore deeper about machine learning, models, optimization, etc, a really good source is the scikit-learn user guide

3.1 Basic exploratory machine learning analysis

This is a pipeline that can be used to compare different models and decide which one is the best for your case and further optimize it and test it on our test data set.

3.1.1 Explore different models

First, you need to import your data, separate features (absorbances) and the target variable, and split the data set into train and test set.

```
# Import data
df = pd.read_csv("./data/UV_pilot.csv")

# Split features and target
X = df.loc[:,"4000":"403"]
y = df.loc[:,"Exposed"]

# Split into train and test sets

X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True, stratify=y, test_s
```

> **Important**
>
> Be sure you stratify your split by using the stratify parameter. This is to make sure the different classes of the target varibale are equality distribute in the split

You need to create a dictionary of the models you want to test. For this example, we are going to use 3 models: Logistic regression, Support Vector Machine, Random Forest but you can add more models if needed.

```
model_dict = {
    "LR": LogisticRegression(),
    "SVM": SVC(),
    "RF": RandomForestClassifier(),
}
```

It is good practice to use pipelines to avoid data leakage especially, if we are using preprocessing algorithms in our data. Usually, we apply standard scaling, so for this, we will build a pipeline that includes the preprocessing algorithm and the model. Click here if you want to learn more about pipelines and how to use them.

Then, we create a for loop which is going to iterate between each model, do a cross validation with our specified pipeline, append the results into a list and print a message with the accuracy and the standard deviation of each model.

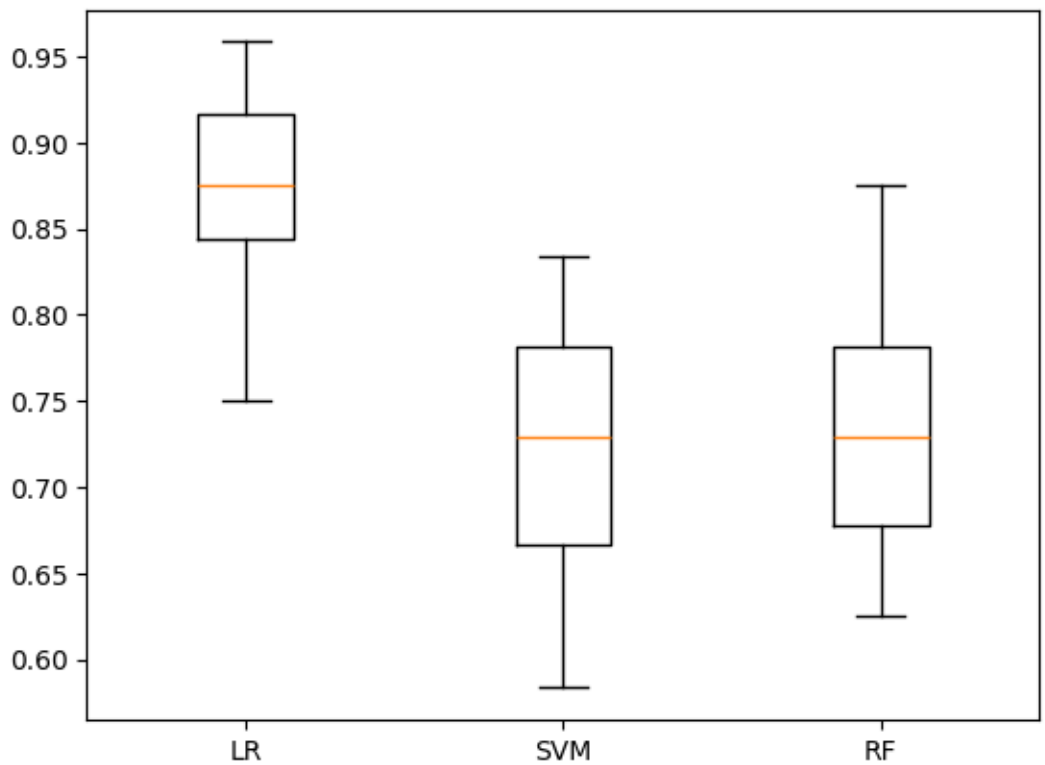
```
results = []
names = []
sss = StratifiedShuffleSplit(n_splits=10, test_size=0.3, random_state=7)

for key, model in model_dict.items():
    pipe = Pipeline([('scaler', StandardScaler()), ('model', model)])

    cv_results = cross_val_score(pipe, X_train, y_train, cv=sss, scoring="accuracy")
    results.append(cv_results)
    names.append(key)
    print(f'Accuracy using {key} is {cv_results.mean():.2f} ± {cv_results.std():.2f}')
```

```
Accuracy using LR is 0.87 ± 0.07
Accuracy using SVM is 0.72 ± 0.08
Accuracy using RF is 0.74 ± 0.07
```

We can examine our results more with a plot

```
fig, ax = plt.subplots()
ax.boxplot(results)
ax.set_xticklabels(names)
```

```
[Text(1, 0, 'LR'), Text(2, 0, 'SVM'), Text(3, 0, 'RF')]
```



From the plot, we can see that logistic regression has the highest accuracy. So, we choose it as our model to further optimize it.

3.1.2 Model optimization

Model optimization consist on finding the best combination of our model's hyperparameter values that will give us the best accuracy. For this, we need to know which hyperparameters can be change, what values they accept and how the affect the model.

For logistic regression we can try different values of C, the type of solver it uses and the type of penalty. We create a dictionary with the hyperparameters and the different values we cant to try.

```
# create a dictionary with different hyperparameters and the values you want to test

param_grid = {'model__penalty': ['l1', 'l2'],
                  'model__C': [0.01, 0.1, 1, 10],
                  'model__solver': ['liblinear', 'saga']}
```

```
# create the new pipleine with the model we choose to optimize
pipe = Pipeline([('scaler', StandardScaler()), ('model', LogisticRegression(max_iter=10000
```

Sklearn in-built function **GridsearchCV** will go over each of the possible combinations between the different values of each hyperparameter and calculate the accuracy of each combination. Then, it will chose the model with the best accuracy and the parameters that uses.

```
search = GridSearchCV(pipe, param_grid, n_jobs=2)
search.fit(X_train, y_train)
print(f"Best parameter (CV score={search.best_score_:.2f})")
print(search.best_params_)
```

```
Best parameter (CV score=0.91)
{'model__C': 1, 'model__penalty': 'l1', 'model__solver': 'saga'}
```

Here, we can see that the best parameters with a cross-validation value of 0.91 are C=10, penalty=l1, solver=liblinear.You can select the best pipleine from the object using .best_estimator_ atrribute.

```
search.best_estimator_
```

```
Pipeline(steps=[('scaler', StandardScaler()),
                ('model',
                 LogisticRegression(C=1, max_iter=10000, penalty='l1',
                                    solver='saga'))])
```

> **Important**
>
> Model optimization can be a extremely long process. Be mindful of the hyperparameters you want to test. This will require to really study the model and make sense of the values you are going to use. Again, scikit-learn documentation is extremely helpful

3.1.3 Test the final model

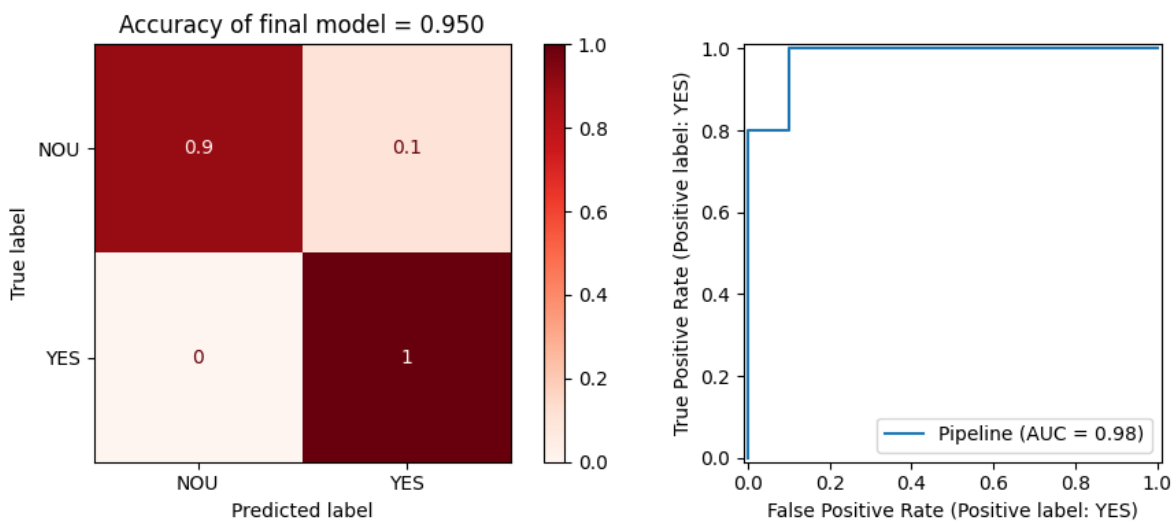
The final optimized model (or in this case pipeline) is tested on the unseen data you split at the beginning of the analysis. Fit the optimized model to your train data set and calculate the predicted values

```
optimized_model = search.best_estimator_
optimized_model.fit(X_train, y_train)
y_pred = optimized_model.predict(X_test)
```

Your final score, confusion matrix and auc-roc curve

```
# Accuracy and confusion  matrix

fig, (ax, ax2) = plt.subplots(1,2, figsize=(10,4), tight_layout=True)


ConfusionMatrixDisplay.from_predictions(y_test, y_pred, normalize='true', cmap='Reds',ax=a

RocCurveDisplay.from_estimator(optimized_model, X_test, y_test, ax=ax2)


title1 = (f'Accuracy of final model = {accuracy_score(y_test, y_pred):.3f}')
ax.set_title(title1)
```

```
Text(0.5, 1.0, 'Accuracy of final model = 0.950')
```



3.2 Nested Cross validation

Nested cross validation can be useful when you do not have a large sample size. Improper use of validation approaches can result in biases in our results. Nested CV and train/test split approaches produce robust and unbiased performance estimates regardless of sample size. Grafical examples of how different validations techniques are shown in Figure 3.1. Nested cross-validation has a inner and outer layer. The inner layer will be used to optimize the model and the outer layer will be used to test the optimized model. This process will be repeated n times. Bear in mind that this is a very time and source consuming approach. You can read more on which different approaches you can use to deal with small data sets [Vabalas et al. (2019)](Varma and Simon 2006). For this example we are going to use nested cross validation as shown in Figure 3.1. We take all the data, and we split it in n folds (outer layer), n-1 folds will be used for model development/optimization (inner layer) and the reminder fold will be used to validate the model. This process will be repeated for each fold in the outer layer. The final accuracy will be the mean of each outer layer.

Validation methods. A: Train/Test Split. B: K-Fold CV. C: Nested CV. D: Partially nested CV. ACC—overall accuracy of the model, ACCi.—accuracy in a single CV fold (Taken from (Vabalas et al. 2019))

As we assumed that we do not have enough samples in our data set, therefore, we do not need to split the data into train and test sets.

```
# Split features and target
X = df.loc[:,"4000":"403"]
y = df.loc[:,"Exposed"]
```

We set up the conditions of our nested cross validations. We add our model and any preprocessing, the number of splits for the inner and outer layer and our final pipline. This time we are using KFold and 3 folds for both layers

```
scaler = StandardScaler()
model = LogisticRegression(max_iter=10000)
splits_outer = 3
splits_inner = 3

pipe = Pipeline(steps=[("scaler", scaler), ("model", model)])

# configure nested cross-validation layers
cv_outer = KFold(n_splits=splits_outer, shuffle=True, random_state=123)
cv_inner = KFold(n_splits=splits_inner, shuffle=True, random_state=123)
```

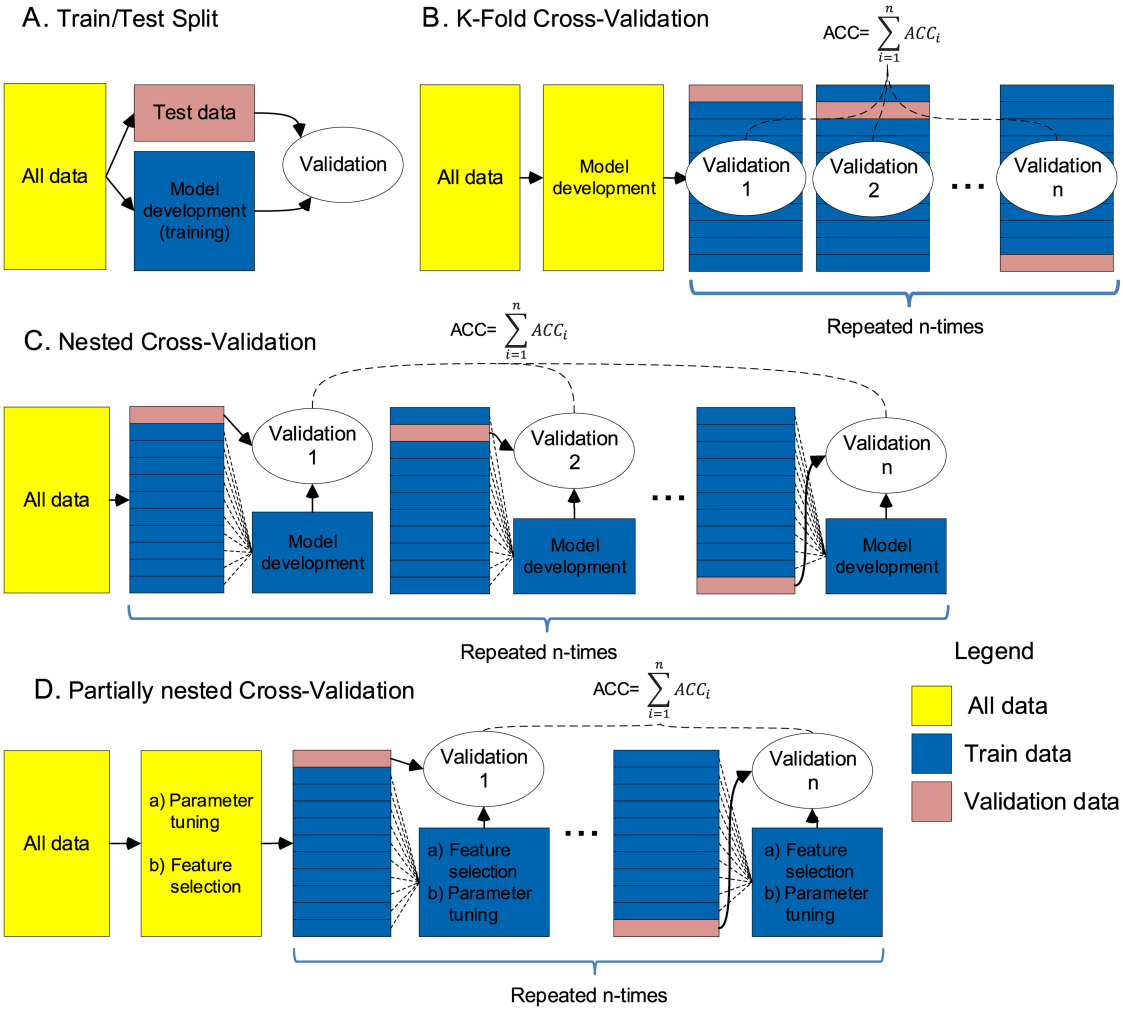


Figure 3.1: validation-techniques

Now, the for loop. First, we split the data using the outer split,with `for train_ix, test_ix in cv_outer.split(X):`. We run GridSearchCV using the train test with the inner layer cross validation (specified in the parameter cv of GridSearchCV). The best modelis then tested on the test set of the outer layer.

```
# create confusion matrix list to save each of external cv layer
cm_nested = []

# enumerate splits and create AUC plot
yhat_nested = []
y_test_nested = []
X_test_nested = []
best_estimators = []
mean_fpr = np.linspace(0, 1, 100)
outer_results = list()


for train_ix, test_ix in cv_outer.split(X):
# split data
    X_train, X_test = X.iloc[train_ix, :], X.iloc[test_ix, :]
    y_train, y_test = y[train_ix], y[test_ix]

    # define search space
    param_grid = {'model__penalty': ['l1', 'l2'],
                  'model__C': [0.01, 0.1, 1, 10],
                  'model__solver': ['liblinear', 'saga']
                  }
    # define search

    search = GridSearchCV(pipe, param_grid, scoring='accuracy', cv=cv_inner, refit=True)
    # execute search
    result = search.fit(X_train, y_train)

    # get the best performing model fit on the whole training set
    best_model = result.best_estimator_

    # create pipeline with the best model
    best_pipe = Pipeline(steps=[("scaler", scaler), ("best_model", best_model)])
    best_pipe.fit(X_train, y_train)
    # evaluate model on the hold out dataset

    yhat = best_pipe.predict(X_test)
```

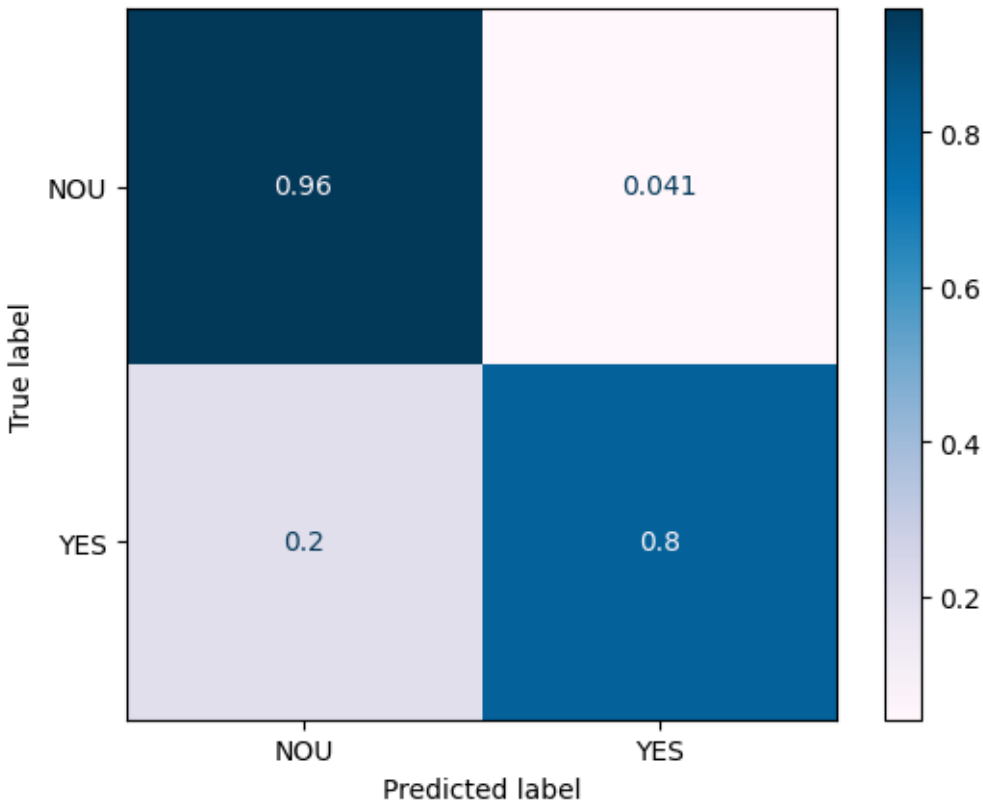
```
    # evaluate the model
    acc = accuracy_score(y_test, yhat)
    cm = confusion_matrix(y_test, yhat)
    yhat_nested.append(yhat)

    y_test_nested.append(y_test)
    X_test_nested.append(X_test)

     # store the result
    outer_results.append(acc)
    cm_nested.append(cm)
    best_estimators.append(best_pipe)

    print(f'Accuracy = {result.best_score_:.2f}, Parameters={result.best_params_}')

# summarize the estimated performance of the model
print(f'Final accuracy = {np.mean(outer_results):.2f} ±  {np.std(outer_results):.2f}')
```

```
Accuracy = 0.92, Parameters={'model__C': 10, 'model__penalty': 'l1', 'model__solver': 'libli
Accuracy = 0.87, Parameters={'model__C': 10, 'model__penalty': 'l2', 'model__solver': 'libli
Accuracy = 0.85, Parameters={'model__C': 10, 'model__penalty': 'l1', 'model__solver': 'libli
Final accuracy = 0.88 ±  0.05
```

To report the result you can use AUC-ROC with confidence intervals or a confusion matrix with the mean of the accuracies of each of the results from the outer layer. Let's plot the confusion matrix

```
# sum all the confusion matrices
cm_sum = np.sum(cm_nested,axis=0)

# we normalised the confusion matrix
cm_sum_normalized = cm_sum.astype('float')/cm_sum.sum(axis=1)[:, np.newaxis]

# Plot the confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm_sum_normalized, display_labels=best_mode
disp.plot(cmap = "PuBu")
```

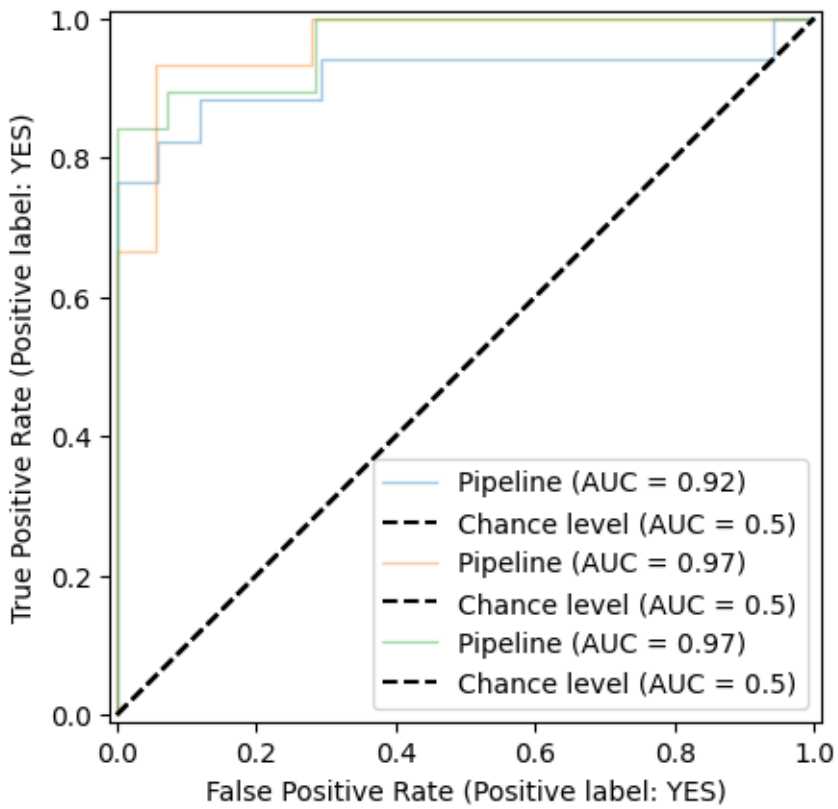
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x15e922240>
```



To plot the AUC-ROC with confidence intervals is a little bit tricky using nested cross validation. We need to create lists where we store the best estimator and the X_train, and y_train sets of each outer layer. Then we run a for loop where we going to plot the roc curve for each X_train, y_test and best model using `RocCurveDisplay.from_estimator` function.

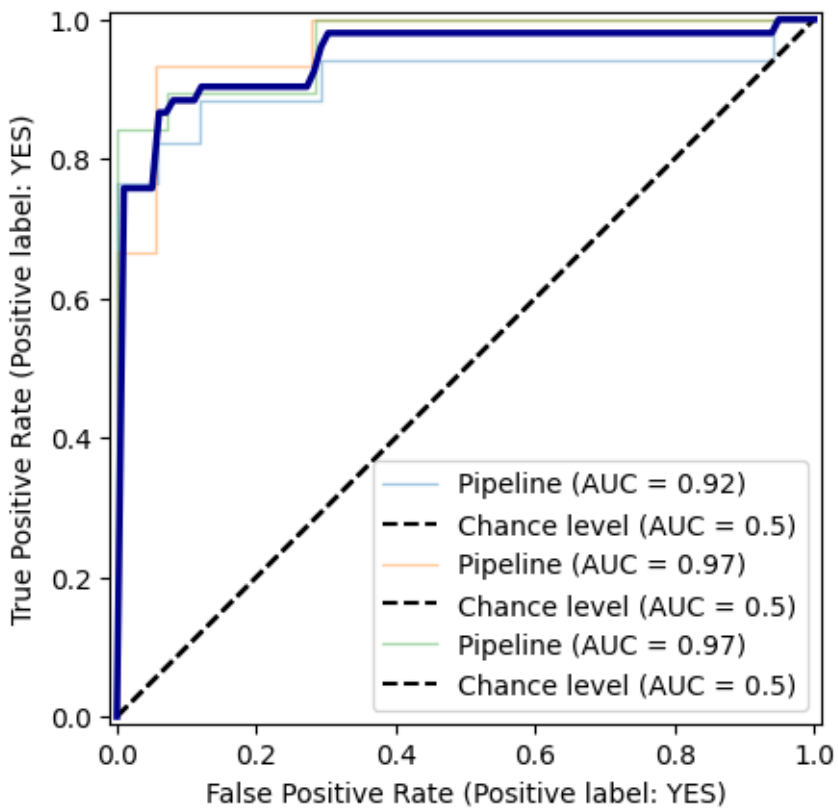
```
# AUC-ROC curve

mean_fpr = np.linspace(0, 1, 100)
tprs = []
aucs = []


fig, ax = plt.subplots()
for a, b, c in zip(X_test_nested,y_test_nested, best_estimators):
        viz = RocCurveDisplay.from_estimator(
        c,
        a,
```

```
b,
alpha=0.4,
lw=1,
plot_chance_level = True,
ax=ax)

interp_tpr = np.interp(mean_fpr, viz.fpr, viz.tpr)
interp_tpr[0] = 0.0
tprs.append(interp_tpr)
aucs.append(viz.roc_auc)
```



We can see three curves that correspond to each outer layer and we can see the values of AUC for each one in the plot legend. Now we need to calculate the mean of the curves plot it over the current plot

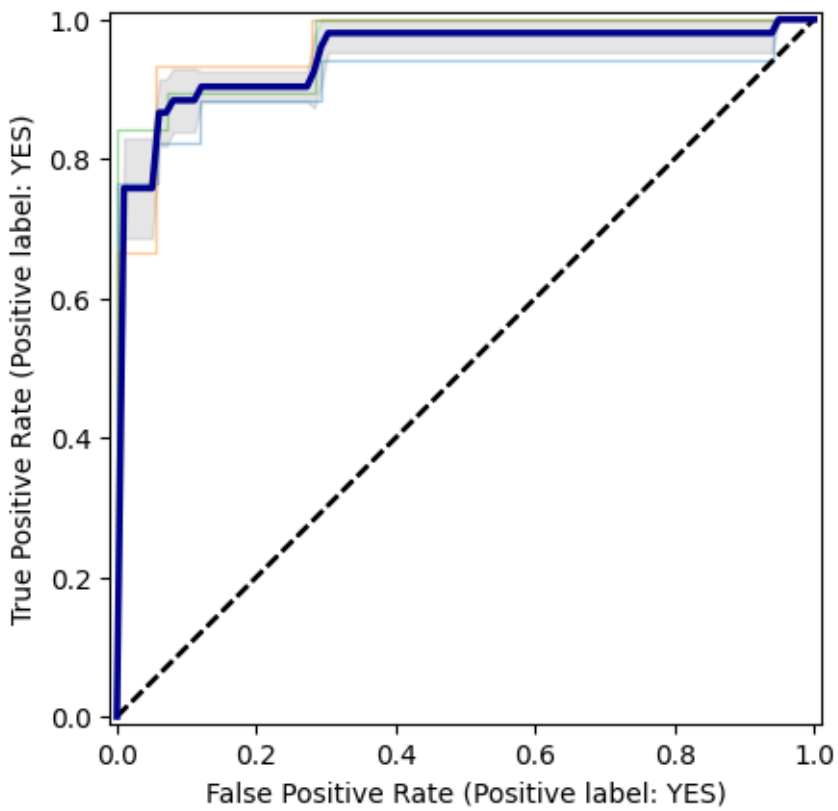
```
mean_tpr = np.mean(tprs, axis=0)
mean_tpr[-1] = 1.0
mean_auc = auc(mean_fpr, mean_tpr)
std_auc = np.std(aucs)

fig, ax = plt.subplots()
for a, b, c in zip(X_test_nested,y_test_nested, best_estimators):
        viz = RocCurveDisplay.from_estimator(
        c,
        a,
        b,
        alpha=0.4,
        lw=1,
        plot_chance_level = True,
        ax=ax)

        interp_tpr = np.interp(mean_fpr, viz.fpr, viz.tpr)
        interp_tpr[0] = 0.0
        tprs.append(interp_tpr)
        aucs.append(viz.roc_auc)

ax.plot(
        mean_fpr,
        mean_tpr,
        color="darkblue",
        label=r"Mean ROC (AUC = %0.2f $\pm$ %0.2f)" % (mean_auc, std_auc),
        lw=2.5,
        alpha=1)
```



Finally, we plot the standard deviation and get rid of the legend

```
fig, ax = plt.subplots()
for a, b, c in zip(X_test_nested,y_test_nested, best_estimators):
        viz = RocCurveDisplay.from_estimator(
        c,
        a,
        b,
        alpha=0.4,
        lw=1,
        plot_chance_level = True,
        ax=ax)

        interp_tpr = np.interp(mean_fpr, viz.fpr, viz.tpr)
        interp_tpr[0] = 0.0
        tprs.append(interp_tpr)
        aucs.append(viz.roc_auc)
```

```
ax.plot(
        mean_fpr,
        mean_tpr,
        color="darkblue",
        label=r"Mean ROC (AUC = %0.2f $\pm$ %0.2f)" % (mean_auc, std_auc),
        lw=2.5,
        alpha=1)

std_tpr = np.std(tprs, axis=0)
tprs_upper = np.minimum(mean_tpr + std_tpr, 1)
tprs_lower = np.maximum(mean_tpr - std_tpr, 0)
ax.fill_between(
        mean_fpr,
        tprs_lower,
        tprs_upper,
        color="grey",
        alpha=0.2)

ax.get_legend().remove()
```



3.3 Nested cross validation plus validation on unseen data

This is a variation of the nested cross validation (develop by our colleagues from IHI). The addition is that you will test the final model in unseen data. So, you can report both, your results of the nested cross-validation as well as the accuracy on the unseen data.

Here we are going to use: 1. Using numpy arrays instead of dataframes 2. Split the data into train and test as a whole 3. Saving a lot of data from the cross validation to further explore the results. 4. Export and save the model (to avoid training every time we run the notebook) 4. Setting up the general aesthetics of the plots at the beginning of the script

For easiness, we have contain the whole code into a function, to avoid mistakes when copy and paste from different notebooks

3.3.1 Set plot aesthetics

Setting up the general aesthetics of the plot. When using seaborn packgaes, you can set the syle, context fonts, of all the plots from the genning

```
sns.set(
    context = "paper",
    style = "white",
    palette = "deep",
    font_scale = 2.0,
    color_codes = True,
    rc = ({"font.family": "Dejavu Sans"})
    )
```

3.3.2 Defining the fuctions

You are going to use two functions: ml_loop and confusion_matrix_mau.

```
# define functions

def ml_loop(
    X_train,
    y_train_species,
    kf,
    num_rounds: int,
    random_grid: dict,
    scoring: str,
    species_model,
    name
):
    """
    Perform a machine learning loop with cross-validation and randomized grid search.

    This function performs a machine learning loop that includes cross-validation,
    randomized grid search for hyperparameter tuning, and model evaluation. It
    trains and evaluates the model multiple times (specified by num_rounds) and
    collects the results.

    Parameters
    ----------
    X_train : array-like of shape (n_samples, n_features)
```

```
    The training input samples.
y_train_species : array-like of shape (n_samples,)
    The true labels for the training samples.
kf : cross-validation generator
    A cross-validation generator, such as KFold or StratifiedKFold.
num_rounds : int
    The number of rounds to run the loop.
random_grid : dict
    The hyperparameter grid for randomized search.
scoring : str
    The scoring metric to evaluate the model, e.g., 'accuracy'.
species_model : estimator object
    The machine learning model to be used, e.g., an instance of SVC.

Returns
-------
species_predicted : list
    A list of predicted labels for the test samples across all rounds.
species_true : list
    A list of true labels for the test samples across all rounds.
kf_results : pd.DataFrame
    A DataFrame containing the model parameters and global accuracy scores.
kf_per_class_results : list
    A list of per-class accuracy scores for


"""
start = time()
kf_results = pd.DataFrame() # model parameters and global accuracy score
kf_per_class_results = []
species_predicted, species_true = [], []

for vuelta in range(num_rounds):
    print(f"\nThis is round {vuelta+1}...\n")
    SEED = np.random.randint(0, 81478)

    # Before your cross-validation loop
    # y_species_train = np.where(y_species_train == 'AA', 0, 1)

    # Inside your cross-validation loop, this step is not needed as the data is alread
```

```
# cross validation and splitting of the validation set
for train_index, test_index in kf.split(X_train, y_train_species):
    X_train_set, X_test_set = X_train[train_index], X_train[test_index]
    y_train_species_set, y_val_species_set = (
        y_train_species[train_index],
        y_train_species[test_index],
    )

    # print('The shape of X train set : {}'.format(X_train_set.shape))
    # print('The shape of y train species : {}'.format(y_train_species_set.shape))
    # print('The shape of X test set : {}'.format(X_test_set.shape))
    # print('The shape of y test species : {}\n'.format(y_val_species_set.shape))

    # generate models using all combinations of settings

    # RANDOMSED GRID SEARCH
    # Random search of parameters, using 5 fold cross validation,
    # search across 100 different combinations, and use all available cores

    n_iter_search = 10
    rsCV = RandomizedSearchCV(
        verbose=1,
        estimator=species_model,
        param_distributions=random_grid,
        n_iter=n_iter_search,
        scoring=scoring,
        cv=kf,
        refit=True,
        n_jobs=-1,
    )

    rsCV_result = rsCV.fit(X_train_set, y_train_species_set)

    # print out results and give hyperparameter settings for best one
    means = rsCV_result.cv_results_["mean_test_score"]
    stds = rsCV_result.cv_results_["std_test_score"]
    params = rsCV_result.cv_results_["params"]
    # for mean, stdev, param in zip(means, stds, params):
    # print("Accuracy of %.2f $\pm$(%.2f) with: %r" % (mean, stdev, param))

    # print best parameter settings
```

```
print(
    "Best accuracy: %.2f using %s"
    % (rsCV_result.best_score_, rsCV_result.best_params_)
)

# Insert the best parameters identified by randomized grid search into the bas
species_classifier = species_model.set_params(**rsCV_result.best_params_)

# Fit your models
species_classifier.fit(X_train_set, y_train_species_set)

# predict test instances
sp_predictions = species_classifier.predict(X_test_set)

# zip all predictions for plotting averaged confusion matrix
# species
for predicted_sp, true_sp in zip(sp_predictions, y_val_species_set):
    species_predicted.append(predicted_sp)
    species_true.append(true_sp)

# species local confusion matrix & classification report
local_cm_species = confusion_matrix(y_val_species_set, sp_predictions)
local_report_species = classification_report(
    y_val_species_set, sp_predictions
)

# append feauture importances
# local_feat_impces_species = pd.DataFrame(species_classifier.feature_importan
#                                     index = features.columns).sort_values(by

# summarizing results
local_kf_results = pd.DataFrame(
    [
        (
            "Accuracy_species",
            accuracy_score(y_val_species_set, sp_predictions),
        ),
        ("TRAIN", str(train_index)),
        ("TEST", str(test_index)),
        ("CM", local_cm_species),
        ("Classification report", local_report_species),
```

```
                    ("y_test", y_val_species_set),
                    # ("Feature importances", #local_feat_impces_species.to_dict())
                ]
            ).T

            local_kf_results.columns = local_kf_results.iloc[0]
            local_kf_results = local_kf_results[1:]
            kf_results = pd.concat(
                [kf_results, local_kf_results], axis=0, join="outer"
            ).reset_index(drop=True)

            # per class accuracy
            local_support = precision_recall_fscore_support(
                y_val_species_set, sp_predictions
            )[3]
            local_acc = np.diag(local_cm_species) / local_support
            kf_per_class_results.append(local_acc)
    elapsed = time() - start
    print(f"\nTime elapsed: {elapsed:.2f} seconds")

    filename = "./results/models/trained_model_" + name + ".sav"
    joblib.dump(species_classifier, filename)

    return species_true, species_predicted, kf_results

def confusion_matrix_mau(y_pred, y_true, xticks_rotation=None, ax=None):
    """
    Displays a normalized confusion matrix using the true and predicted labels.

    This function generates and displays a normalized confusion matrix using the
    true labels (`y_true`) and the predicted labels (`y_pred`). The confusion
    matrix is displayed using a grayscale colormap and is normalized to show
    proportions instead of raw counts.

    Args:
        y_pred (array-like of shape (n_samples,)): The predicted labels.
        y_true (array-like of shape (n_samples,)): The true labels.
        ax (matplotlib.axes.Axes, optional): The axes on which to plot the confusion matri
            If None, the current axes will be used.

    Returns:
```

```
        None
    """
    ConfusionMatrixDisplay.from_predictions(y_pred=y_pred, y_true=y_true,normalize='true',
```

The new thing is the use of joblib to save our trained model. The principle is the same as nested cross validation.

3.3.3 Split the data

We load our data and split into train and test set. We also stratify the split using the "Exposed" variable and then separate our features and target from each of the splits

```
X_train = np.asarray(train_set.iloc[:,5:-1]) # feature matrix
y_train_exposure = np.asarray(train_set['Exposed'])

X_test = np.asarray(test_set.iloc[:,5:-1])
y_test_species = np.asarray(test_set['Exposed'])
```

Why are we using numpy arrays instead of dataframes? Numpy arrays provide a consistent and standardized data format that scikit-learn classifiers can easily handle. This allows for integration with other scikit-learn functionalities, such as preprocessing, cross-validation, and model evaluation

> **Tip**
>
> Sklearn classifiers can easyly handle numpy arrays

3.3.4 Setting the parameters of nested cross validation

Here, we defined the number of folds (inner layer) and the number of rounds (outer layer). Note that we are using a random number generator as random_seed. This number will change each round and will give use a different split in the outer layer

```
# Set validation procedure
num_folds = 5 # split training set into 5 parts for validation
num_rounds = 2 # increase this to 5 or 10 once code is bug-free
seed = 42 # pick any integer. This ensures reproducibility of the tests
random_seed = np.random.randint(0, 81478)

scoring = 'accuracy' # score model accuracy
```

```
# cross validation strategy
kf = KFold(
        n_splits = num_folds,
        shuffle = True,
        random_state = random_seed
        )
```

3.3.5 Setting the parameters of nested cross validation

The hyperparameters for support vector machine that we are going to test are C, kernel and gamma.

```
random_grid = {'C': [10, 1.0, 0.1, 0.01], 'kernel': ["linear", 'rbf', 'poly'], 'gamma': [0

species_model = SVC()
```

3.3.6 Run the loop

We run the loop. The function needs X and y train, the split type, number of rounds and folds, hyperparameters, the score and the model

```
# Species prediction

y_mau_true, y_mau_predicted, main_results = ml_loop(X_train, y_train_exposure, kf, num_rou
```

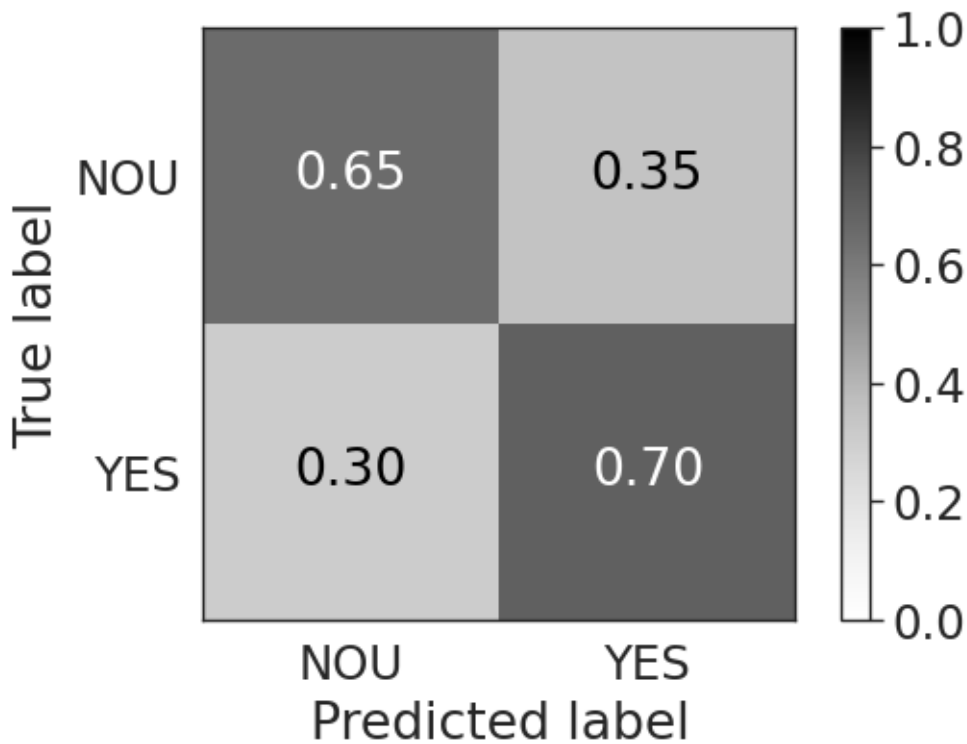
```
This is round 1...

Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.78 using {'kernel': 'poly', 'gamma': 10, 'C': 10}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.75 using {'kernel': 'poly', 'gamma': 10, 'C': 1.0}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.86 using {'kernel': 'poly', 'gamma': 1, 'C': 10}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.77 using {'kernel': 'poly', 'gamma': 10, 'C': 0.1}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.74 using {'kernel': 'poly', 'gamma': 1, 'C': 1.0}
```

```
This is round 2...

Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.78 using {'kernel': 'poly', 'gamma': 10, 'C': 1.0}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.75 using {'kernel': 'poly', 'gamma': 1, 'C': 10}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.86 using {'kernel': 'poly', 'gamma': 10, 'C': 0.01}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.77 using {'kernel': 'poly', 'gamma': 10, 'C': 10}
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best accuracy: 0.74 using {'kernel': 'poly', 'gamma': 1, 'C': 1.0}

Time elapsed: 3.40 seconds
```

You can see how each round will contain 5 best models.

We plot a confusion matrix which is the sum of all the confusion matrices from each round.

```
# this function make pretty confusion matrices
confusion_matrix_mau(y_true=y_mau_true, y_pred=y_mau_predicted)
```

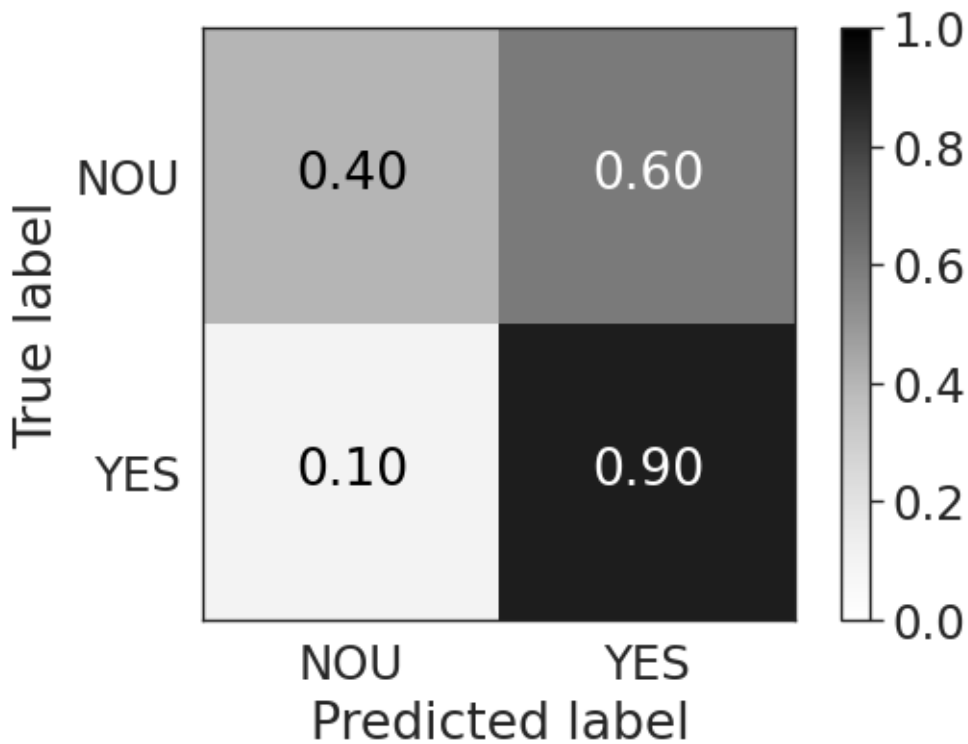


We load our trained model, test it in the unseen data and plot the confusion matrix

```
loaded_model_exposure = joblib.load('./results/models/trained_model_mau_mau.sav')
```

```
y_hd_pred = loaded_model_exposure.predict(X_test)
acc = accuracy_score(y_true=y_test_species, y_pred=y_hd_pred)
print(f"Accuracy on test set using the head: {acc}")

confusion_matrix_mau(y_true=y_test_species, y_pred=y_hd_pred)
```

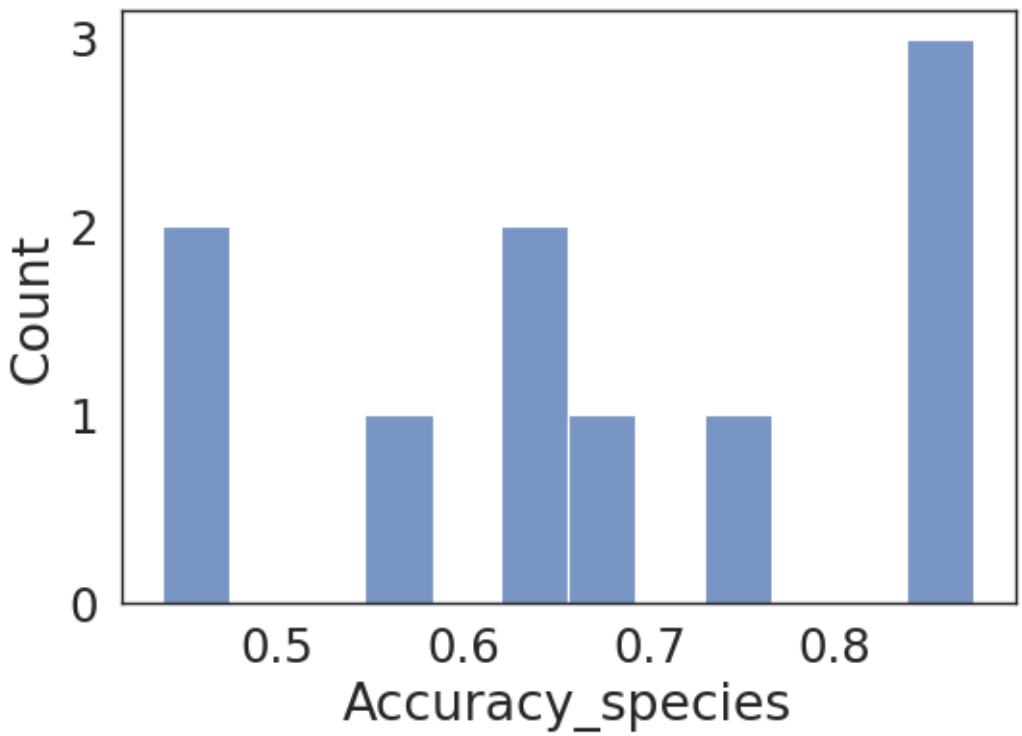
```
Accuracy on test set using the head: 0.65
```



Since we save a lot of data from the loop, for example accyracy of each inner layer. We can plot an histogram to check how stable was the accuracy across layers.

```
sns.histplot(data=main_results, x='Accuracy_species',bins=12)
```

```
<Axes: xlabel='Accuracy_species', ylabel='Count'>
```


3
2
1
0
Count
0.5
0.6
0.7
0.8
Accuracy_species

4 A closer look into spectral quality

dfdfsdfds

References

González Jiménez, Mario, Simon A. Babayan, Pegah Khazaeli, Margaret Doyle, Finlay Walton, Elliott Reedy, Thomas Glew, et al. 2019. "Prediction of Mosquito Species and Population Age Structure Using Mid-Infrared Spectroscopy and Supervised Machine Learning [Version 3; Peer Review: 2 Approved]." *Wellcome Open Research*. https://doi.org/10.12688/wellcomeopenres.15201.3.

Pazmiño-Betancourth, Mauro, Ivan Casas Gómez-Uribarri, Karina Mondragon-Shem, Simon A Babayan, Francesco Baldini, and Lee Rafuse Haines. 2024. "Advancing Age Grading Techniques for Glossina Morsitans Morsitans, Vectors of African Trypanosomiasis, Through Mid-Infrared Spectroscopy and Machine Learning." *Biology Methods and Protocols*, August, bpae058. https://doi.org/10.1093/biomethods/bpae058.

Pazmiño-Betancourth, Mauro, Victor Ochoa-Gutiérrez, Heather M. Ferguson, Mario González-Jiménez, Klaas Wynne, Francesco Baldini, and David Childs. 2023. "Evaluation of Diffuse Reflectance Spectroscopy for Predicting Age, Species, and Cuticular Resistance of Anopheles Gambiae s.l Under Laboratory Conditions." *Scientific Reports* 13 (1): 18499. https://doi.org/10.1038/s41598-023-45696-x.

Siria, Doreen J., Roger Sanou, Joshua Mitton, Emmanuel P. Mwanga, Abdoulaye Niang, Issiaka Sare, Paul C. D. Johnson, et al. 2022. "Rapid Age-Grading and Species Identification of Natural Mosquitoes for Malaria Surveillance." *Nature Communications* 13 (1): 1–9. https://doi.org/10.1038/s41467-022-28980-8.

Vabalas, Andrius, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. 2019. "Machine Learning Algorithm Validation with a Limited Sample Size." *PLoS ONE* 14 (11): e0224365. https://doi.org/10.1371/journal.pone.0224365.

Varma, Sudhir, and Richard Simon. 2006. "Bias in Error Estimation When Using Cross-Validation for Model Selection." *BMC Bioinformatics* 7 (1). https://doi.org/10.1186/1471-2105-7-91.